

- Regularization techniques are a family of methods that reduce the generalization gap between training & test performance.

↳ Involves adding explicit terms to the loss function that favor certain parameter choices

↳ Commonly refers to any strategy that improves generalization.

9.1: Explicit Regularization:

- Consider a model $f(\vec{x}, \theta)$ using a training set $\{x_i, y_i\}$ of input/output pairs.

- We seek parameters $\hat{\theta}$ that minimize the loss $L[\theta]$:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} L[\theta]$$

$$= \underset{\theta}{\operatorname{argmin}} \left[\sum_{i=1}^I L_i[\vec{x}_i, \vec{y}_i] \right]$$

↳ This term measures the mismatch between network predictions & output targets for each training pair.

- To bias this minimization towards certain solutions, we include an additional term:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \left[\sum_{i=1}^I L_i[\vec{x}_i, \vec{y}_i] + \lambda \cdot g[\theta] \right]$$

A positive scalar that controls the relative contribution of the original loss function & the regularization term.

↳ $g[\cdot]$ is a function that returns a scalar which takes larger values when the parameters are less preferred.

- The minima of the regularized loss function usually differ from those of the original, so the training procedure converges to different parameter values.

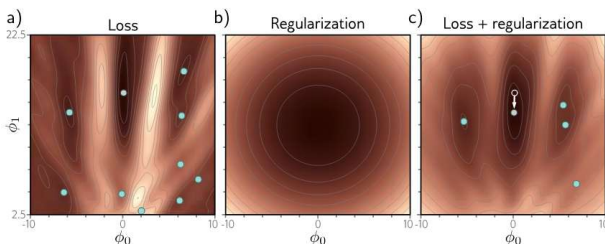


Figure 9.1 Explicit regularization. a) Loss function for Gabor model (see section 6.1.2). Cyan circles represent local minima. Gray circle represents the global minimum. b) The regularization term favors parameters close to the center of the plot by adding an increasing penalty as we move away from this point. c) The final loss function is the sum of the original loss function plus the regularization term. This surface has fewer local minima, and the global minimum has moved to a different position (arrow shows change).

Probabilistic Interpretation:

- Regularization can be viewed from a Probabilistic Perspective.
- Loss functions are constructed from the maximum likelihood criterion:

- Regularization can be viewed from a probabilistic perspective.
- Loss functions are constructed from the maximum likelihood criterion:

$$\hat{\theta} = \arg \max_{\theta} \left[\prod_{i=1}^I \Pr(\bar{y}_i | \bar{x}_i, \theta) \right]$$

- The regularization term can be viewed as a prior $\Pr(\theta)$ that represents knowledge about the parameters before we observe the data. We now have a Maximum a Posteriori (MAP) criterion:

$$\hat{\theta} = \arg \max_{\theta} \left[\prod_{i=1}^I \Pr(\bar{y}_i | \bar{x}_i, \theta) \Pr(\theta) \right]$$

- Moving back to NLL, we see that: $\lambda \cdot g[\theta] = -\log[\Pr(\theta)]$

L2 Regularization:

- Which solutions should the regularization term penalize?
- The most commonly used regularization term is the L2 norm.
- ↳ It penalizes the sum of squares of the parameter values:

$$\hat{\theta} = \arg \min \left[\sum_{i=1}^I L_i(\bar{x}_i, \bar{y}_i) + \lambda \cdot \sum_j \theta_j^2 \right]$$

↳ j indexes the parameters

- Known as Ridge / Frobenius norm regularization.

- For neural nets, L2 regularization is usually applied to the weights, but not biases.
- ↳ So it's referred to as the "weight decay" term.

- The effect of L2 regularization is to encourage smaller weights, so the output function is smoother.

- Intuition: The output prediction is a weighted sum of the activations at the last hidden layer. If the weights have less magnitude, the output will vary less. The same logic applies to the computations of the pre-activation at the last hidden layer. (And so on...)

- When λ is small, it has little effect. When λ gets bigger, the fit to the data becomes less accurate; the function gets smoother.
- ↳ This might improve test performance for 2 reasons:

1) If the network is overfitting, then adding regularization means the network must trade adherence to the data against the desire to be smooth. The error due to variance reduces at the cost of increased bias.

2) When the network is over-parameterized, some of the extra model capacity describes areas with no training data. So regularizing will favor functions that smoothly interpolate between nearby data points.

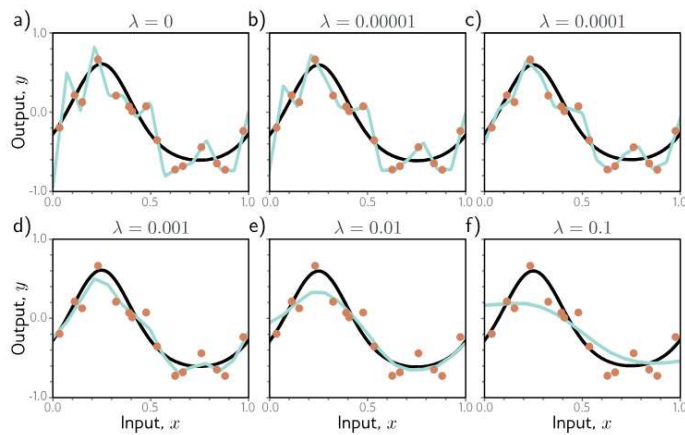


Figure 9.2 L2 regularization in simplified network with 14 hidden units (see figure 8.4). a-f) Fitted functions as we increase the regularization coefficient λ . The black curve is the true function, the orange circles are the noisy training data, and the cyan curve is the fitted model. For small λ (panels a-b), the fitted function passes exactly through the data points. For intermediate λ (panels c-d), the function is smoother and more similar to the ground truth. For large λ (panels e-f), the regularization term overpowers the likelihood term, so the fitted function is too smooth and the overall fit is worse.

9.2: Implicit Regularization:

- We can prefer some solutions over others without an explicit regularization term.
↳ Known as implicit regularization.
- Neither GD / SGD move naturally to the minimum of the loss function.

Implicit Regularization in Gradient Descent:

- Consider a continuous version of gradient descent where the step size is infinitesimal.
- The change in parameters θ is governed by:

$$\frac{d\theta}{dt} = - \frac{\partial L}{\partial \theta}$$

- Gradient descent approximates this process with a series of discrete steps of size α :

$$\theta_{b+1} \leftarrow \theta_b - \alpha \cdot \frac{\partial L}{\partial \theta}$$

- This discretization causes a deviation from the continuous path.

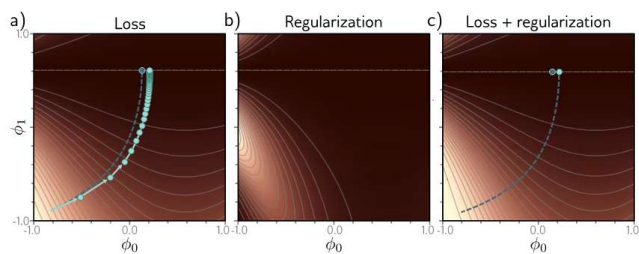


Figure 9.3 Implicit regularization in gradient descent. a) Loss function with family of global minima on horizontal line $\phi_1 = 0.61$. Dashed blue line shows continuous gradient descent path starting in bottom-left. Cyan trajectory shows discrete gradient descent with step size 0.1 (first few steps shown explicitly as arrows). The finite step size causes the paths to diverge and reach a different final position. b) This disparity can be approximated by adding a regularization term to the continuous gradient descent loss function that penalizes the squared gradient magnitude. c) After adding this term, the continuous gradient descent path converges to the same place that the discrete one did on the original function.

- The deviation can be understood by deriving a modified loss term $\tilde{L}$ for the continuous case that arrives at the same place as the discretized version on the original loss L .

↳ This modified loss is:

$$\tilde{L}_{GD}[\theta] = L[\theta] + \frac{\alpha}{4} \left\| \frac{\partial L}{\partial \theta} \right\|^2$$

↳ The discrete trajectory is repelled from places where the gradient norm is large.

- This doesn't change the positions of the minima. But it does change the loss surface elsewhere.
! thus the trajectory. So it may converge to a different minimum.

- Implicit regularization in GD may explain why full batch GD generalizes better with large step sizes.

Implicit Regularization in Stochastic Gradient Descent:

- A similar analysis can be done for SGD.

- The modified loss function can be derived as:

$$\begin{aligned} \tilde{L}_{SGD}[\theta] &= \tilde{L}_{GD}[\theta] + \frac{\alpha}{4B} \sum_{b=1}^B \left\| \frac{\partial L_b}{\partial \theta} - \frac{\partial L}{\partial \theta} \right\|^2 \\ &= L[\theta] + \frac{\alpha}{4} \left\| \frac{\partial L}{\partial \theta} \right\|^2 + \frac{\alpha}{4B} \sum_{b=1}^B \left\| \frac{\partial L_b}{\partial \theta} - \frac{\partial L}{\partial \theta} \right\|^2 \end{aligned}$$

↳ Loss for the b^{th} of the B batches in an epoch

- * Both L & L_b now represent the means of the I individual losses in the full dataset & the $|B|$ individual losses in the batch, respectively:

$$L = \frac{1}{I} \sum_{i=1}^I L_i[\bar{x}_i, y_i] \quad \& \quad L_b = \frac{1}{|B|} \sum_{i \in B_b} L_i[\bar{x}_i, y_i]$$

- The extra regularization term corresponds to the variance of the gradients of the batch losses L_b .
- SGD implicitly favors places where the gradients are more stable.
- The modified loss does not change the position of the global minimum.

• SGD implicitly favors flatter minima where the gradient is small...

• The modified loss does not change the position of the global minimum.

• SGD generalizes better than gradient descent.

↳ Smaller batches generally perform better than larger ones.

↳ While one explanation is that the random process lets it reach different areas of the loss function...

another is that it is due to implicit regularization, which encourages solutions where all the data fits well

[so batch variance is small] rather than solutions where some of the data fits very well ; Others

less well [maybe same loss, but with larger batch variance]

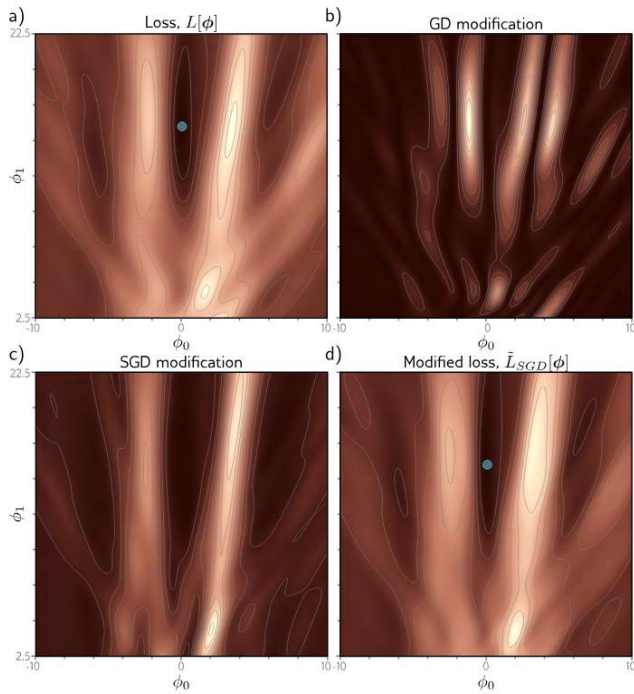


Figure 9.4 Implicit regularization for stochastic gradient descent. a) Original loss function for Gabor model (section 6.1.2). Blue point represents global minimum. b) Implicit regularization term from gradient descent penalizes the squared gradient magnitude. c) Additional implicit regularization from stochastic gradient descent penalizes the variance of the batch gradients. d) Modified loss function (sum of original loss plus two implicit regularization components). Blue point represents global minimum which may now be in a different place from panel (a).

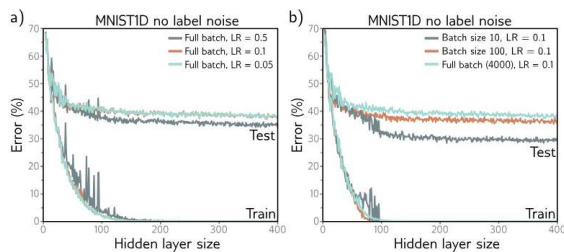


Figure 9.5 Effect of learning rate (LR) and batch size for 4000 training and 4000 test examples from MNIST-1D (see figure 8.1) for a neural network with two hidden layers. a) Performance is better for large learning rates than for intermediate or small ones. In each case, the number of iterations is 6000/LR, so each solution has the opportunity to move the same distance. b) Performance is superior for smaller batch sizes. In each case, the number of iterations was chosen so that the training data were memorized at roughly the same model capacity.

9.3: Heuristics to Improve Performance:

• There are other heuristic methods to improve performance

1.0 • Heuristics to Improve Performance

- There are other heuristic methods to improve performance.

Early Stopping:

- Stopping the training procedure before the model fully converged.

↳ could reduce overfitting if the model has already learned the broad shape of the underlying function, but hasn't had the time to overfit to the noise.

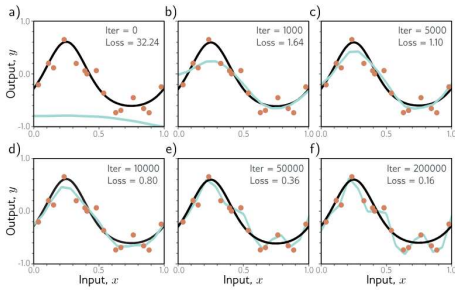


Figure 9.6 Early stopping. a) Simplified shallow network model with 14 linear regions (figure 8.4) is initialized randomly (cyan curve) and trained with SGD using a batch size of five and a learning rate of 0.05. b-d) As training proceeds, the function first captures the coarse structure of the true function (black curve) before e-f) overfitting to the noisy training data (orange points). Although the training loss continues to decrease throughout this process, the learned models in panels (c) and (d) are closest to the true underlying function. They will generalize better on average to test data than those in panels (e) or (f).

- Since the weights are initialized to small values, they don't have time to become large.

↳ similar effect to L2 regularization

- Another view: It reduces the effective model capacity.

↳ we move down the bias-variance tradeoff curve, ∴ performance improves.

- Has 1 hyperparameter: The number of steps after which training is terminated.

↳ Chosen empirically, using a validation set.

↳ This can be selected by training once! The performance on the validation set is monitored every T iterations, ∴ the associated parameters are stored.

↳ The stored parameters where validation performance is best are chosen.

Ensembling:

- Building several models ∴ average their predictions.

↳ A group of such models is known as an ensemble.

↳ Improves test performance, at the cost of training more models/storing them/performing inference on them.

- They can be combined by taking the mean of the outputs [for regression problems] or the mean of the pre-softmax activations [classification problems].

↳ The assumption is that model errors are independent ∴ will cancel out.

- Alternative way to group: take the median of the outputs [regression] or the most frequent predicted class [classification]

- Two approaches to training:

- 1) Use different random initializations. May help in regions of input space far from the data.
The fitted functions are relatively unconstrained, so different models may give different predictions.
The average of those models may generalize better than one single model.

- 2) Generate several different datasets by re-sampling the training data w/ replacement : training a different model on each. Known as "bootstrap aggregating" / "bagging". Has the effect of smoothing out the data.
If a data point is not present in one training set, the model will interpolate from nearby points.
This smooths out outliers in the data.

- 3) Could change hyperparams, or train a completely different family of models.

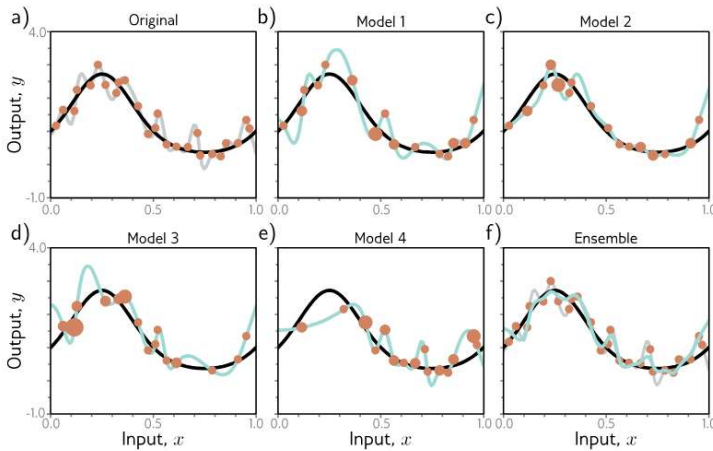


Figure 9.7 Ensemble methods. a) Fitting a single model (gray curve) to the entire dataset (orange points). b-e) Four models created by re-sampling the data with replacement (bagging) four times (size of orange point indicates number of times the data point was re-sampled). f) When we average the predictions of this ensemble, the result (cyan curve) is smoother than the result from panel (a) for the full dataset (gray curve) and will probably generalize better.

Dropout:

- Dropout clamps a random subset (typically 50%) of hidden units to 0 at each iteration of SGD.
↳ makes the network less dependent on any given hidden unit, encourages the weights to have smaller magnitudes so the change in the function due to the presence/absence of any specific hidden unit is reduced.

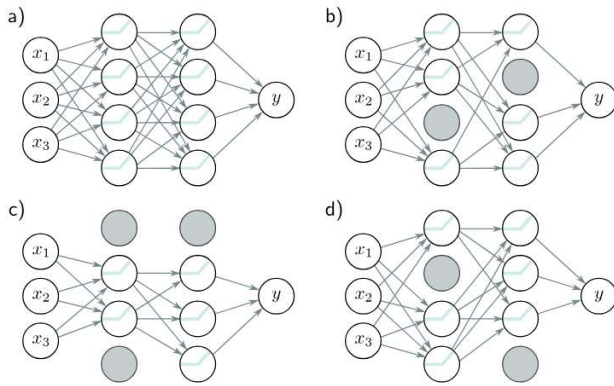


Figure 9.8 Dropout. a) Original network. b-d) At each training iteration, a random subset of hidden units is clamped to zero (gray nodes). The result is that the incoming and outgoing weights from these units have no effect, so we are training with a slightly different network each time.

- It can eliminate "kinks" in the function that are far from the training data ; don't affect the loss.
 - ↳ consider 3 hidden units that become active sequentially as we move along the curve.
 - ↳ The 1st causes a large increase in slope. The second decreases it, ; the third cancels out the decrease.
 - ↳ when several units act this way, eliminating one causes a considerable change to the output function in the half-space where that unit was active.
 - ↳ SGD will try to compensate for the change, ; the dependencies will be eliminated over time.
- Overall effect: Large, unnecessary changes between training data points are gradually removed, even though they contribute nothing to the loss.

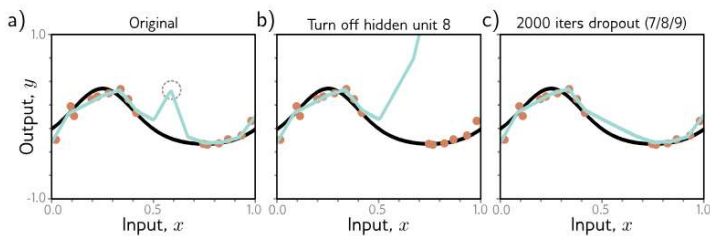


Figure 9.9 Dropout mechanism. a) An undesirable kink in the curve is caused by a sequential increase in the slope, decrease in the slope (at circled joint), and then another increase to return the curve to its original trajectory. Here we are using full-batch gradient descent, and the model (from figure 8.4) fits the data as well as possible, so further training won't remove the kink. b) Consider what happens if we remove the eighth hidden unit that produced the circled joint in panel (a), as might happen using dropout. Without the decrease in the slope, the right-hand side of the function takes an upwards trajectory, and a subsequent gradient descent step will aim to compensate for this change. c) Curve after 2000 iterations of (i) randomly removing one of the three hidden units that cause the kink and (ii) performing a gradient descent step. The kink does not affect the loss but is nonetheless removed by this approximation of the dropout mechanism.

- At test time, we run the network as usual with all hidden units active.
 - ↳ But the network now has more hidden units than it was trained with at any given iteration.
 - ↳ So we multiply the weights by $(1 - \text{dropout probability})$ to compensate. Known as the weight scaling inference rule.
- A different approach is to use Monte Carlo Dropout.
 - ↳ Run the network multiple times with different random subsets of units clamped to 0, then combine results.
 - ↳ Closely related to ensembling since we have a different model each time, but we don't need to train/store multiple networks.

Applying Noise:

- Dropout can be interpreted as applying noise to the network activations.
- There are other ways of adding noise to other parts of the network during training to make the final model more robust:

- 1) Add noise to the input data. This smooths out the learned function.
- 2) Add noise to the weights. This encourages the network to make sensible predictions even for small perturbations of the weights. Result is that the training converges to a wide, flat, local minima where changing individual weights does not matter much.
- 3) Perturb the labels. Since maximum likelihood criteria for multiclass classification aims to predict the class with absolute certainty, this makes the final activations very large / very small for correct / incorrect respectively. We can discourage this overconfident behavior by assuming that a portion p of the training labels are incorrect, & belong w/ equal probability to other classes. This is done by randomly changing the labels at each training iteration.
OR: Change the loss function to minimize the cross entropy between the predicted distribution & a distribution where the true label has probability $1-p$, & other classes have equal probability. $\rightarrow$ Known as label smoothing!

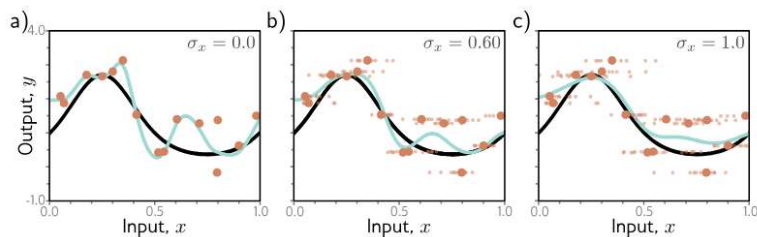


Figure 9.10 Adding noise to inputs. At each step of SGD, random noise with variance σ_x^2 is added to the batch data. a-c) Fitted model with different noise levels (small dots represent ten samples). Adding more noise smooths out the fitted function (cyan line).

Bayesian Inference:

- Maximum likelihood is generally overconfident.

$\rightarrow$ However, many parameters may be broadly compatible with the data & only slightly less likely.

$\rightarrow$ The Bayesian approach treats these parameters as unknown variables & computes a distribution $Pr(\theta | \mathcal{E}\bar{x}_i, \bar{y}_i; \mathcal{Z})$ over these parameters θ (conditioned on the training data $\mathcal{E}\bar{x}_i, \bar{y}_i; \mathcal{Z}$) using Bayes Rule:

$$Pr(\theta | \mathcal{E}\bar{x}_i, \bar{y}_i; \mathcal{Z}) = \frac{\prod_{i=1}^I Pr(\bar{y}_i | x_i, \theta) Pr(\theta)}{\int \prod_{i=1}^I Pr(\bar{y}_i | x_i, \theta) Pr(\theta) d\theta}$$

$\rightarrow$ The Prior Probability of the Parameters.

$\rightarrow$ Denominator is a normalizing term.

- So every Parameter choice is assigned a Probability.

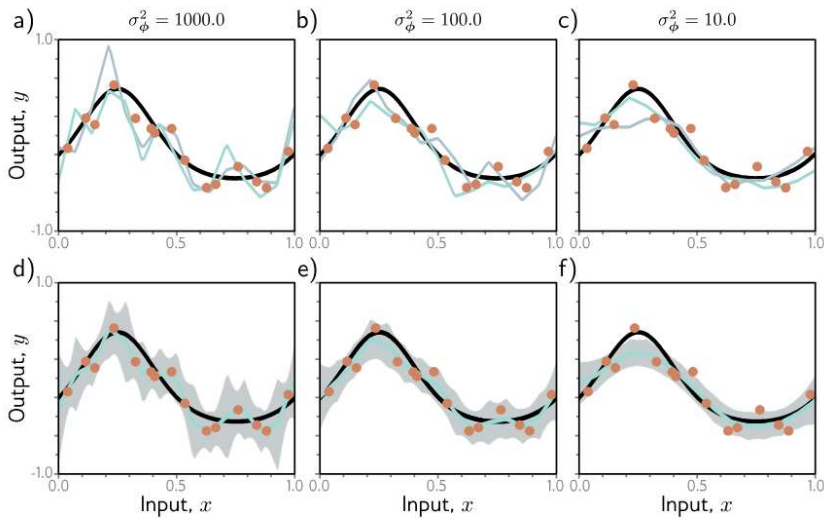


Figure 9.11 Bayesian approach for simplified network model (see figure 8.4). The parameters are treated as uncertain. The posterior probability $Pr(\phi|\{x_i, y_i\})$ for a set of parameters is determined by their compatibility with the data $\{x_i, y_i\}$ and a prior distribution $Pr(\phi)$. a-c) Two sets of parameters (cyan and gray curves) sampled from the posterior using normally distributed priors with mean zero and three variances. When the prior variance σ_ϕ^2 is small, the parameters also tend to be small, and the functions smoother. d-f) Inference proceeds by taking a weighted sum over all possible parameter values where the weights are the posterior probabilities. This produces both a prediction of the mean (cyan curves) and the associated uncertainty (gray region is two standard deviations).

- The Prediction $\bar{y}$ for new input $\bar{x}$ is an integral of the Predictions for each Parameter Set, where the weights are the associated Probabilities:

$$Pr(\bar{y}|\bar{x}, \mathcal{E}\bar{x}, \bar{y}; \mathcal{Z}) = \int Pr(\bar{y}|\bar{x}, \phi) Pr(\phi|\mathcal{E}\bar{x}, \bar{y}; \mathcal{Z}) d\phi$$

* Effectively an infinite weighted ensemble. The weights depend on:

- i) The prior Probability of the Parameters
- ii) Their agreement with the data.

- The Bayesian approach suffers from the inability to practically represent the full Probability distribution over the Parameters or to integrate over it during inference.

- All current Methods make approximations, usually adding complexity to learning/ inference.

Transfer Learning ; Multi-Task Learning:

- When training data is limited, other datasets can be exploited to improve Performance.
- In transfer learning, the network is pre-trained to perform a related Secondary task for which data is more Plentiful.
↳ The resulting model is then adapted to the Original task.

- In transfer learning, the network is pre-trained to perform a related secondary task for which data is more plentiful.
 - ↳ The resulting model is then adapted to the original task.
 - ↳ Usually done by removing the last layer; adding one or more layers that produce a suitable output.
 - ↳ The main model may be fixed; the new models are trained for the task. OR we can fine-tune the entire model.
- The idea is that the network will build a good internal representation of the data from the secondary task, which can be exploited for the original task.
- Equivalently, can be viewed as initializing most of the parameters of the final network in a sensible part of the space likely to produce a good solution.
- Multi-task learning is a related technique where the network is trained to solve several problems concurrently.
 - ↳ Ex: Network takes an image; simultaneously learn to segment the scene, estimate pixel-wise depth; predict a caption describing the image. Model performance may improve when learned simultaneously.

Self-Supervised Learning:

- Transfer/Multi-task learning assume we have a lot of data for a secondary task / data for multiple tasks to be learned simultaneously.
- If not, we can create large amounts of "free" labeled data using self-supervised learning; use this for transfer learning.
- 2 families of methods for self-supervised learning:

1) Generative:

- ↳ Part of each data example is masked; the secondary task is to predict the missing part.
- ↳ Ex: Might use a corpus of unlabeled images; a secondary task that aims to inpaint (fill in) missing parts of the image. OR might use a large corpus of text; mask some words. We train the network to predict the missing words, then fine-tune it for a language task we are interested in.

2) Contrastive:

- ↳ Pairs of examples with commonalities are compared to unrelated pairs.
 - ↳ For images the secondary task might be to identify if a pair of images are transformed versions of one another or are unconnected.
 - ↳ For text, the secondary task might be to determine if two sentences followed one another in the original document.

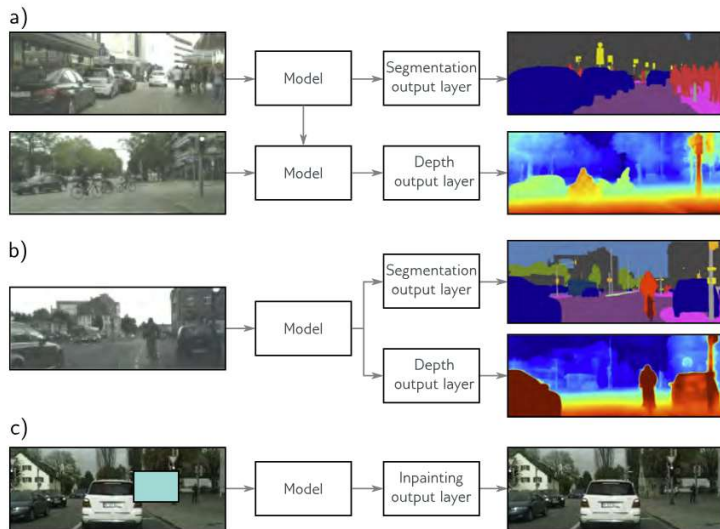


Figure 9.12 Transfer, multi-task, and self-supervised learning. a) Transfer learning is used when we have limited labeled data for the primary task (here depth estimation) but plentiful data for a secondary task (here segmentation). We train a model for the secondary task, remove the final layers, and replace them with new layers appropriate to the primary task. We then train only the new layers or fine-tune the entire network for the primary task. The network learns a good internal representation from the secondary task that is then exploited for the primary task. b) In multi-task learning, we train a model to perform multiple tasks simultaneously, hoping that performance on each will improve. c) In generative self-supervised learning, we remove part of the data and train the network to complete the missing information. Here, the task is to fill in (inpaint) a masked portion of the image. This permits transfer learning when no labels are available. Images from Cordts et al. (2016).

Augmentation:

- Transfer learning improves performance by exploiting a different dataset.
- Multi-task learning improves performance using additional labels.
- A third option is to expand the dataset:
 - ↳ Transform each input in such a way that the label stays the same.
 - ↳ For images:
 - Could rotate, blur, flip, etc.
 - The label remains valid.
 - ↳ For text:
 - Substitute synonyms / translate to another language & back again.
 - ↳ For audio:
 - Amplify / attenuate different frequency bands.
- This is known as data augmentation.
 - ↳ The goal is to teach the model to be indifferent to these transformations.

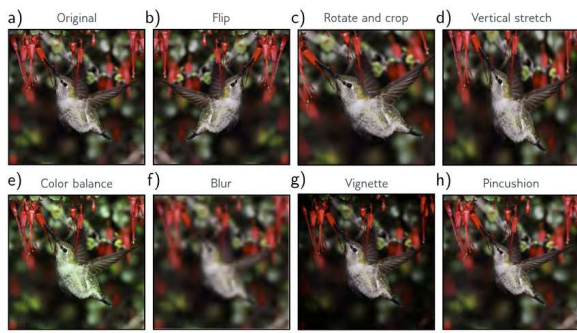


Figure 9.13 Data augmentation. For some problems, each data example can be transformed to augment the dataset. a) Original image. b-h) Various geometric and photometric transformations of this image. For image classification, all these images still have the same label, "bird." Adapted from Wu et al. (2015a).

9.4: Summary:

- Explicit regularization adds an extra term to the loss function that changes the position of the minimum.
 - ↳ Can be interpreted as a prior probability over the parameters.
- GD / SGD with Pinik step size does not naturally descend to the minimum of the loss function.
 - ↳ This bias can be interpreted as adding additional terms to the loss function.
 - ↳ This is implicit regularization.

- There are many heuristics for improving generalization.
- There are 4 main principles behind the methods above:
 - 1) Encourage the function to be smoother
 - 2) Increase the amount of data
 - 3) Combine models
 - 4) Search for wider minima.

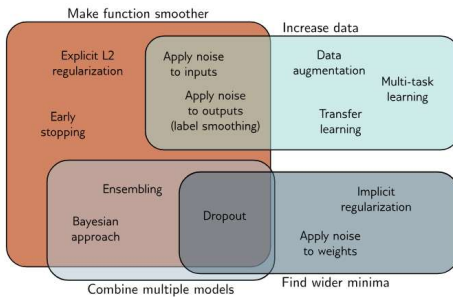


Figure 9.14 Regularization methods. The regularization methods discussed in this chapter aim to improve generalization by one of four mechanisms. Some methods aim to make the modeled function smoother. Other methods increase the effective amount of data. The third group of methods combine multiple models and hence mitigate against uncertainty in the fitting process. Finally, the fourth group of methods encourages the training process to converge to a wide minimum where small errors in the estimated parameters are less important (see also figure 20.11).

- Another way to improve generalization is to choose the model architecture to suit the task.